Vericoding

Verification-Driven Development

From behavioural features to machine-checked proofs

Gavin Mendel-Gleason · Scidonia · July 2026

Vibecoding

The status quo of LLM-driven development.

You prompt. The model emits. It runs.

What you leave behind:

- **No specification** of intended behaviour
- **No evidence** of correctness
- **No conformance regime** for the next generation

The only durable artifact is the code itself.

Vericoding

The methodology specsaver operationalises.

You write a specification. The spec survives.

What you leave behind:

- **Features** — Gherkin scenarios domain experts can review
- **Contracts** — mathematical pre/post-conditions
- **Proof obligations** — machine-checked in Rocq/Coq

Generated code is a replaceable implementation detail.

Two Development Flows

Retrospective

- **Feature** — Gherkin scenario tables
- **Implementation** — write the production code
- **Testing (sanity)** — make sure it basically works
- **Contract** — capture observed behaviour as mathematical spec
- **Testing (dialectic)** — re-run under contract checking; either contract or implementation is wrong — the dialectic
- **Formal Proof** — lower to Coq, LLM closes obligations; DISPROVED → back to dialectic

Contract-First

- **Feature** — Gherkin scenario tables
- **Contract** — write spec as acceptance criteria
- **Implementation** — build code against the specification
- **Testing** — validate that implementation satisfies contract
- **Formal Proof** — machine-checked against kernel

Both converge: the contract is the lasting artifact.
DISPROVED yields witnesses → back to the dialectic.

Contract Language

The pure terminating fragment of Python

Every clause is a boolean-valued lambda:

- `requires(state, args)` — admissibility
- `ensures(state, args, result, new_state)` — success post
- `when(state, args)` — exception conditions
- `invariant(state)` — global legality

Static purity check rejects side effects, loops, and non-boolean returns before execution.

Anatomised as mathematical record

$$C = \langle \Sigma,\ args,\ \textbf{pre},\ \textbf{post},\ \mathcal{X},\ \mathcal{I},\ \mathcal{D},\ \mathcal{W},\ \mathcal{R},\ \mathcal{G} \rangle$$

$\mathcal{X}$ — exceptional exits (when + ensures + frame)

$\mathcal{I}$ — invariants on every state

$\mathcal{D}$ — derived-state definitions

$\mathcal{W}$ / $\mathcal{R}$ — semantic frame

$\mathcal{G}$ — ghost state

Contract Anatomy: Implementation

A simple inventory reserve function against SQLAlchemy:

```
def reserve(self, engine, sku, order_id, quantity):
    with engine.begin() as conn:
        row = conn.execute(
            text("SELECT on_hand, reserved FROM products"
                 " WHERE sku = :sku"), {"sku": sku}
        ).fetchone()

        if row is None: raise ProductNotFoundError(sku)

        on_hand, reserved = row
        available = on_hand - reserved

        if available < quantity:
            raise InsufficientStockError(sku, quantity, available)

        conn.execute(
            text("UPDATE products SET reserved = reserved + :qty"
                 " WHERE sku = :sku"),
            {"qty": quantity, "sku": sku},
        )

    return ReservationReceipt(sku=sku, quantity=quantity)
```

Contract Anatomy: Specification

The same operation, specified once:

```
reserve_contract = Contract(
    requires=[
        lambda s, a: a.quantity > 0,
        lambda s, a: a.sku in s.observed.products,
    ],
    ensures=[
        lambda s, a, r, s2: (
            s2.products[a.sku].reserved == s.products[a.sku].reserved + a.quantity),
    ],
    exceptions=[
        ExcExit(raises=InsufficientStockError,
                when=[lambda s, a:
                    s.products[a.sku].on_hand - s.products[a.sku].reserved < a.quantity],
                writes={"state.failure_log"}),
    ],
    writes={"state.products[sku].reserved", "state.failure_log"},
    reads={"state.products[sku].on_hand", "state.products[sku].reserved"},
)
```

requires
admissibility

ensures
what changes

exceptions
when + outcome

writes/reads
semantic frame

Symmetric Projection

The commuting diagram. One projection `snap`, applied before and after execution. Contracts compare the pair.

What snap projects. `snap : Context → SpecState` reads the concrete world — SQLAlchemy queries → row dicts, event log → typed tuples, pure aggregation → derived fields — and returns a frozen, comparable, purely functional value. This lifts raw mutable state into the semantic domain where contracts are stated.

Why symmetry matters. The diagram commutes: `snap(exec(ctx)) = post`, and the same `snap` produced `pre`. Without this guarantee, testing (Python bools) and proving (Coq Props) would reason about different interpretations of state. Symmetry guarantees they see the same thing.

What it gives us. Contracts written once as pure predicates serve double duty: tested on real rows at runtime, lowering to proof obligations over the same declared schema. The specification is the source of truth at both levels.

Symmetry as Bridge

Why do testing and proving agree? Because the state they see is produced by the same function.

Declared schema

The contract's `state_schema` names every observed field, its type, and its provenance. The runtime snapshot and the Coq state model are both derived from this single declaration.

Semantic frame

`writes` and `reads` are declarative paths over the schema. The runtime checker and the frame-soundness proof enforce the same footprint.

Same snapshot

One function, called twice. No divergent "read" and "check" path. The frame, derived-consistency, and invariant checkers all operate over this shared interpretation.

The specification is the bridge. Write it once; test it on data today; prove it against the kernel tomorrow. The lift is automatic.

Symmetric Projection — Runner

1. materialize
witness → temp SQLite DB + engine + EventLog

2. pre-check
invariants on pre-state; admissibility (requires)

3. execute
service runs against real SQLite via SQLAlchemy; wrapper emits typed events into the log

4. frame check
everything outside writes unchanged

5. derived check
derived fields ≡ recomputed from observed

6. post-check
ensures (or exit ensures) for the outcome; invariants on post-state

Domain Declaration

One object → runners, CLI, conformance suite.

```
inventory = SqlDomain(
    name         = "inventory",
    package      = "examples.inventory",
    materializer = SqlMaterializer(ddl, TableSpec[]),
    projection   = SqlProjection(types, hooks),
    operations   = [
        SqlOperation(contract, wrapper,
                     witness_builder, feature_file),
        ...
    ]
)
```

scenario runners | CLI dispatch | conformance tests

Adding a domain: types + service + contracts + features + witness builders + one declaration. Everything else (materializer, projection, runner wiring, test dispatch) is derived.

Theory Stack

Services run **unmodified** on differentially-tested stubs.

Service ordinary SQLAlchemy / logging / OTel API code

Shim make_engine, make_log_capture, make_otel_capture

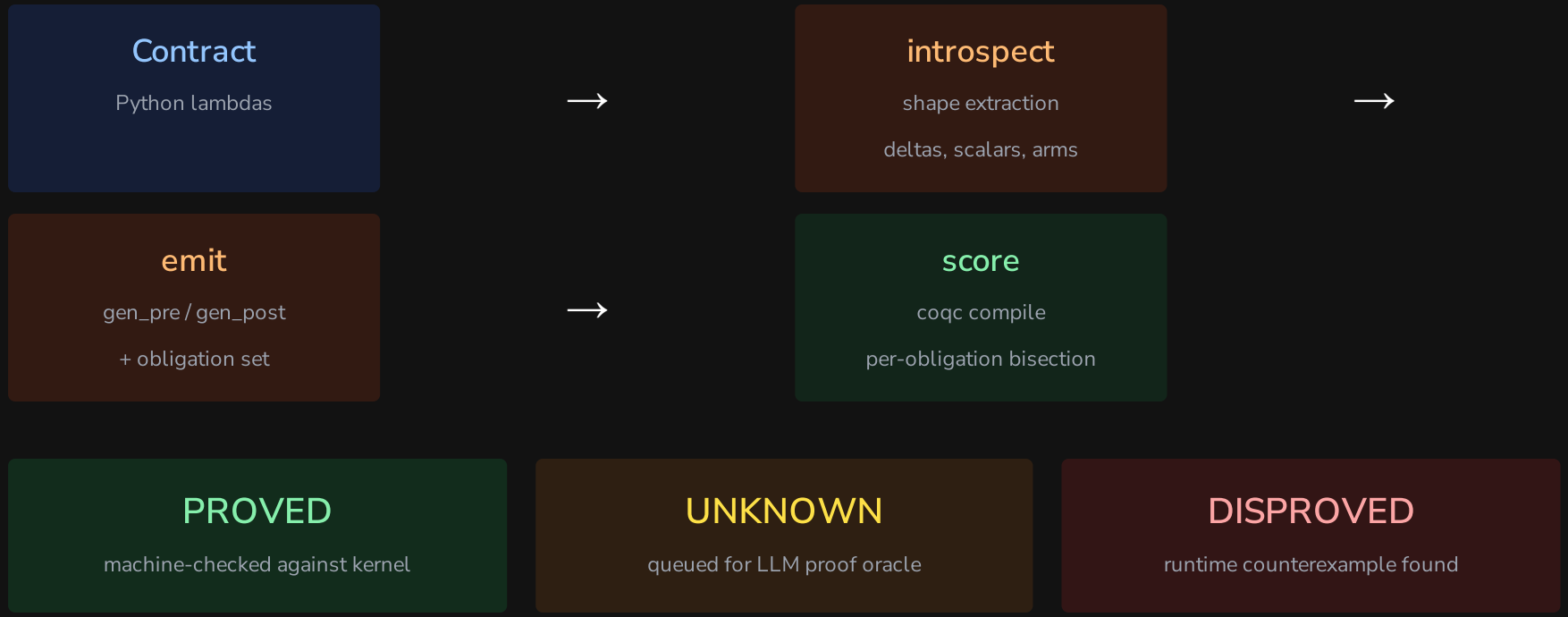
StubHandler operational semantics over the state model

State model TableStore / LogStore / OtelStore

Trace final interpretation (events)

Theory conformance: Stub and real library agree observationally on a differential suite; translator rejects out-of-fragment input loudly; state model and trace are pure data.

Lowering Pipeline



Example: Inventory Reserve

Pre-condition:

$$\mathsf{pre}(s, a) \;=\; a.quantity > 0 \;\wedge\; a.sku \in s.products$$

Post-condition:

$$\mathsf{post}(s, a, r, s') \;=\; s'.reserved = s.reserved + a.quantity \;\wedge\; \mathsf{extends\_by\_one}(s.res\_log, s'.res\_log, \ldots)$$

Exception exit:

$$\mathcal{X} = \{\, \langle \mathsf{InsufficientStock},\; s.available < a.quantity,\; \{failure\_log\}\rangle \,\}$$

Verified State

Domain	Tables	Ops	Rows	Obligations
inventory	1	3	22	6/6, 6/6, 4/4
bank_transfer	2	1	11	7/7
Total		**6**	**42**	**23/23**

All four store-obligation contracts fully proved in Rocq.

The Vericoding Promise

Vibecoding leaves

code and hope

Vericoding leaves

a specification and evidence